Min Rewards

AlgoExpert · Hard · Arrays

The problem

- Each student gets >= 1 reward; a student scoring higher than an adjacent one gets more.
- Minimum total.
- Two passes (left-to-right, right-to-left).

Intuition

- Each student must beat a lower-scoring neighbour on both sides, so do one pass each way and take the max.

Brute force vs optimal

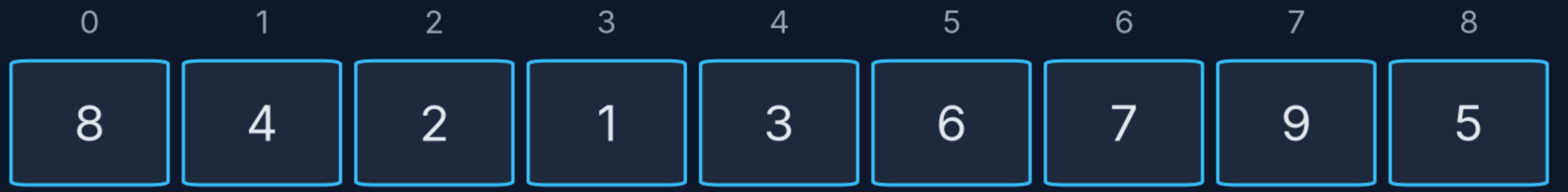
- Brute force: repeat fixes until stable → $O(n^2)$
- Expand from local minima → $O(n)$
- Two passes (left, right) → $O(n)$

Approach

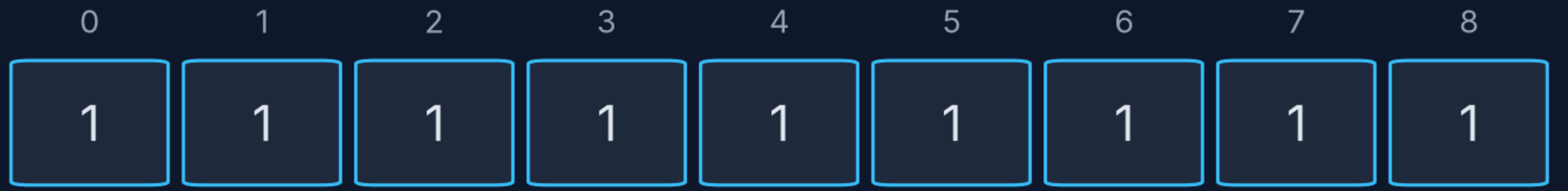
- 1. Everyone starts with one
- 2. Left to right, give one more than a lower left neighbour
- 3. Right to left, ensure one more than a lower right neighbour
- 4. Sum the rewards

Worked example

scores



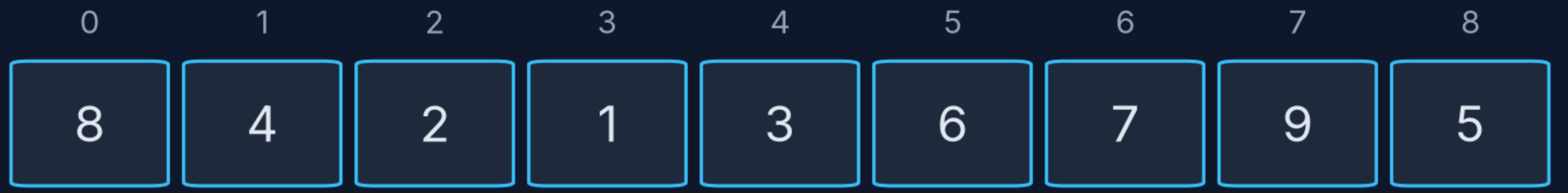
rewards



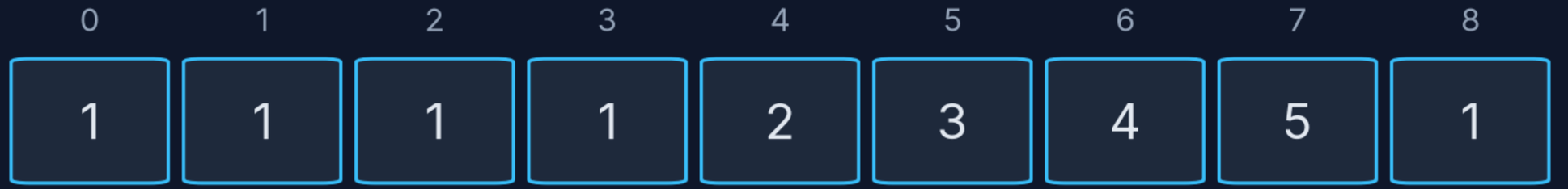
Start: everyone gets 1

Worked example

scores



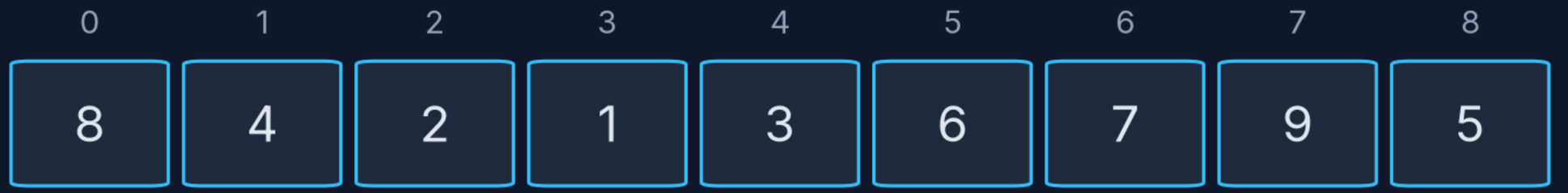
rewards after left→right



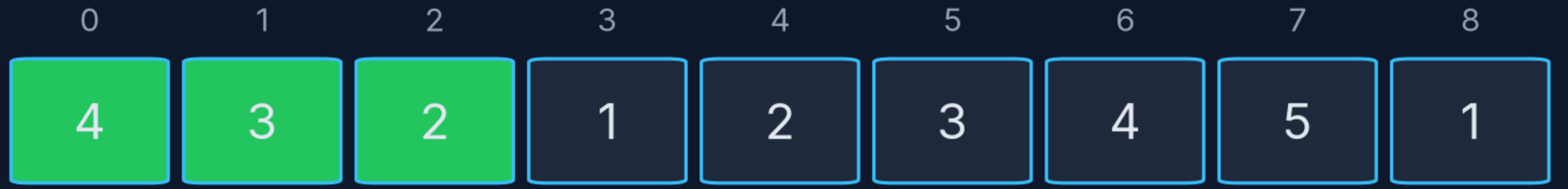
Left pass: beat left neighbour → +1

Worked example

scores

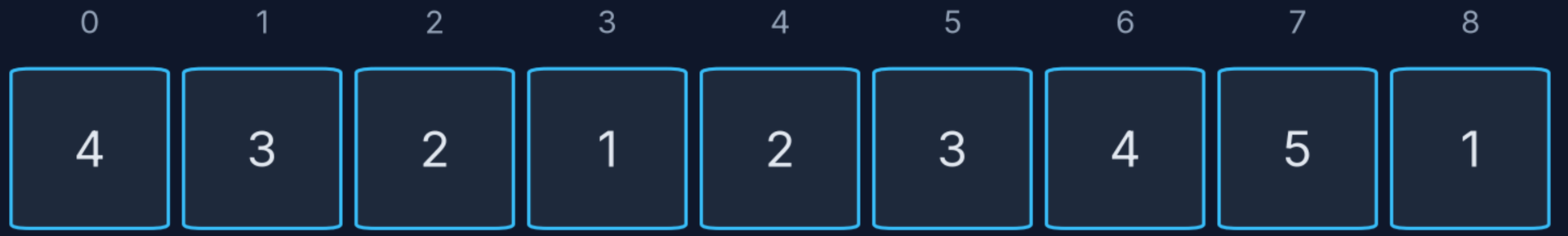


rewards after right→left



Right pass: beat right neighbour → max(own, right + 1)

Worked example




```
sum = 25
```

Total = 25

Java solution

```java
public class MinRewards {
    public static int minRewards(int[] scores) {
        int n = scores.length;
        int[] r = new int[n];
        java.util.Arrays.fill(r, 1);
        for (int i = 1; i < n; i++) if (scores[i] > scores[i - 1]) r[i] = r[i - 1] + 1;
        for (int i = n - 2; i >= 0; i--) if (scores[i] > scores[i + 1]) r[i] = Math.max(r[i], r[i + 1] + 1);
        int sum = 0;
        for (int x : r) sum += x;
        return sum;
    }
}
```

Complexity

- Time: O(n)
- Space: O(n)

Edge cases & pitfalls

- Use max in the second pass
- Scores are unique
- Single student → 1

Interview tips & follow-ups

- Same as LeetCode Candy

Recap

- Each student must beat a lower-scoring neighbour on both sides, so do one pass each way and take the max.
- Complexity: O(n)